

I TÉRMINO 2026

TAREA 2 - SISTEMAS EMBEBIDOS

COMUNICACIÓN ENTRE MICROCONTROLADORES ATMEGA328P - PIC16F887

Ejecutantes:

Juan Francisco Sánchez Maldonado

López Gonzabay Gerson Andrés

Stefano Joel Moreno Mosquera

Joshua Esaud Vélez Montalván

Paralelo 1

Docente

Ing. Ronald Solis Mesa

Junio, 2026

Tabla de contenido

Objetivos del Informe	3
• Objetivo general	3
• Objetivos específicos	3
Descripción técnica completa del juego	4
• Arquitectura y roles de los microcontroladores	4
Simulación Proteus.....	8
Explicación Del Código Del ATMEGA328P y PIC16F887	8
ATmega328P – Controlador Maestro del Juego	8
• Control de la Matriz LED	9
• Generación de Secuencias Aleatorias.....	10
• Lectura de Pulsadores	10
• Comunicación con el PIC16F887	11
• Verificación de la Respuesta del Usuario	11
PIC16F887 – Subsistema Generador de Audio	12
Esquema de Comunicación entre microcontroladores	15
Enlace de Repositorio.....	17
Conclusiones y Recomendaciones	17
• Conclusiones.....	17
• Recomendaciones.....	18

Objetivos del Informe

- **Objetivo general**

Desarrollar un sistema de juego interactivo de 3 niveles de dificultad que combine salidas visuales mediante una matriz LED 8x8 controlada por el microcontrolador ATmega328P, y salidas auditivas mediante un sistema de reproducción de melodías con el microcontrolador PIC16F887, y entradas mediante pulsadores, empleando comunicación entre ambos dispositivos y simulando el funcionamiento completo en Proteus, con el propósito de reforzar habilidades en programación y control de sistemas embebidos. El sistema deberá ser programado en C para ambos microcontroladores.

- **Objetivos específicos**

- Diseñar e implementar un sistema de control de una matriz LED 8x8 con el microcontrolador ATmega328P para desplegar caracteres, símbolos y animaciones del juego.
- Programar un sistema de reproducción de melodías utilizando el PIC16F887, capaz de interpretar comandos recibidos desde el ATmega328P.
- Integrar botones físicos como entradas para interactuar con el juego (selección, acción, reinicio, etc.).
- Implementar 3 niveles de dificultad ya sea modificando la velocidad, lógica o complejidad del juego.
- Establecer un canal de comunicación entre los microcontroladores que permita la sincronización entre efectos visuales y sonoros.
- Simular el juego de manera completa en el entorno Proteus, integrando los dos microcontroladores, la matriz LED y el sistema de audio.
- Documentar todo el proceso de diseño, desarrollo y simulación en un informe técnico, incluyendo repositorio con evidencias y código

Descripción técnica completa del juego

El sistema consiste en la implementación electrónica y programable del clásico juego de memoria conceptual "Simón Dice". El diseño arquitectónico se divide en dos unidades de procesamiento descentralizadas (ATmega328P y PIC16F887) que operan en sincronía mediante un bus de comunicación paralelo unidireccional, garantizando una respuesta en tiempo real sin retardos (lag) entre los estímulos visuales y auditivos.

- **Arquitectura y roles de los microcontroladores**

Microcontrolador Maestro (ATmega328P)

Es el cerebro del sistema. Se encarga de la ejecución de la máquina de estados finitos (FSM) que gobierna el flujo del juego.

Funciones principales:

- **Gestión de Entradas:**

Monitoreo y filtrado por software (debouncing) de los 4 pulsadores que representan las opciones de respuesta del usuario, además del botón de inicio/reinicio.

- **Procesamiento de Datos:**

Generación de secuencias pseudoaleatorias utilizando un generador congruencial basado en el desborde de un temporizador (Timer) para asegurar la aleatoriedad de cada partida.

- **Control Visual (Matriz LED 8x8):**

Control dinámico de la matriz mediante multiplexación temporal. La matriz se divide virtualmente en 4 cuadrantes simétricos de 4x4 LEDs para representar visualmente

los cuatro colores o posiciones del juego. Adicionalmente, genera caracteres alfanuméricos en formato scrolling o estáticos para indicar el nivel actual (L1, L2, L3), la victoria o el fin del juego (GAME OVER).

- **Sincronización:**

Codificación y envío de comandos de audio en paralelo hacia el esclavo inmediatamente antes de ejecutar cualquier acción visual.

Microcontrolador Esclavo (PIC16F887)

Actúa como un coprocesador periférico dedicado exclusivamente a la síntesis de audio analógico.

Funciones principales:

- **Decodificación por Interrupción:**
Monitoreo del bus paralelo. Mediante la interrupción por cambio de estado del Puerto B o por muestreo síncrono por hardware (pin Strobe), lee el comando del bus sin perder ciclos de reloj en bucles de espera.
- Utiliza el módulo CCP1 (Capture/Compare/PWM) configurado en modo PWM para variar la frecuencia del temporizador interno (Timer1/Timer2). Al modificar el periodo del PWM con un ciclo de trabajo (duty cycle) constante del 50%, se generan ondas cuadradas puras que excitan un transductor piezoeléctrico (Buzzer) o altavoz, traduciendo los comandos digitales en notas musicales específicas o alertas acústicas de error.

Explicación Detallada De Los Niveles De Dificultad

El sistema implementa 3 niveles de dificultad que modifican dinámicamente el comportamiento del hardware y la exigencia cognitiva sobre el usuario. El cambio de nivel se realiza tras completar con éxito una longitud de secuencia determinada.

Nivel 1 (Fácil): "Modo Aprendiz"

- **Lógica visual:**

La secuencia se muestra a una velocidad lenta. Cada cuadrante se ilumina durante 800 ms con un intervalo de apagado de 200 ms antes del siguiente paso.

- **Complejidad:**

La longitud máxima de la secuencia para superar el nivel es de 4 pasos.

- **Tolerancia:**

El usuario dispone de tiempo ilimitado para ingresar su respuesta; el sistema espera indefinidamente la pulsación sin penalización temporal.

Nivel 2 (Medio): "Modo Estándar"

- **Lógica visual:**

La velocidad de refresco e iluminación de la secuencia se duplica. El tiempo de encendido del cuadrante disminuye a 400 ms y el intervalo de apagado a 100 ms, exigiendo mayor fijación visual.

- **Complejidad:**

La secuencia máxima se extiende a 6 pasos. Se integran patrones de repetición aleatoria inmediata (por ejemplo, encender el mismo cuadrante dos veces seguidas).

- **Visualización:**

Al iniciar, la matriz borra el estado anterior y despliega mediante un parpadeo de 500 ms el identificador L2.

Nivel 3 (Difícil): "Modo Experto"

Lógica visual:

Destello de alta velocidad. Los cuadrantes se iluminan únicamente por 200 ms con intervalos mínimos de 50 m.

- **Complejidad**

La secuencia requerida para ganar el juego es de 8 pasos consecutivos sin error.

- **Restricción de Tiempo (Timeout):**

Se activa un desborde por temporizador (Timer) en el ATmega328P. Una vez que el sistema cede el turno al usuario, este dispone de una ventana de tiempo máxima de 3 segundos para presionar cada botón de la secuencia. Si el temporizador se desborda antes de que el usuario interactúe, el sistema procesa el evento automáticamente como una entrada incorrecta (Timeout Error), enviando la señal de Game Over (derrota) al PIC16F887 y reiniciando el juego al Nivel 1.

Nivel	Dificultad	Características
Nivel 1	Fácil	Secuencia corta y velocidad de exposición lenta.
Nivel 2	Medio	Secuencia estándar y velocidad intermedia.
Nivel 3	Difícil	Secuencia larga y velocidad de destello rápida.

Tabla 1 - Niveles

Simulación Proteus.

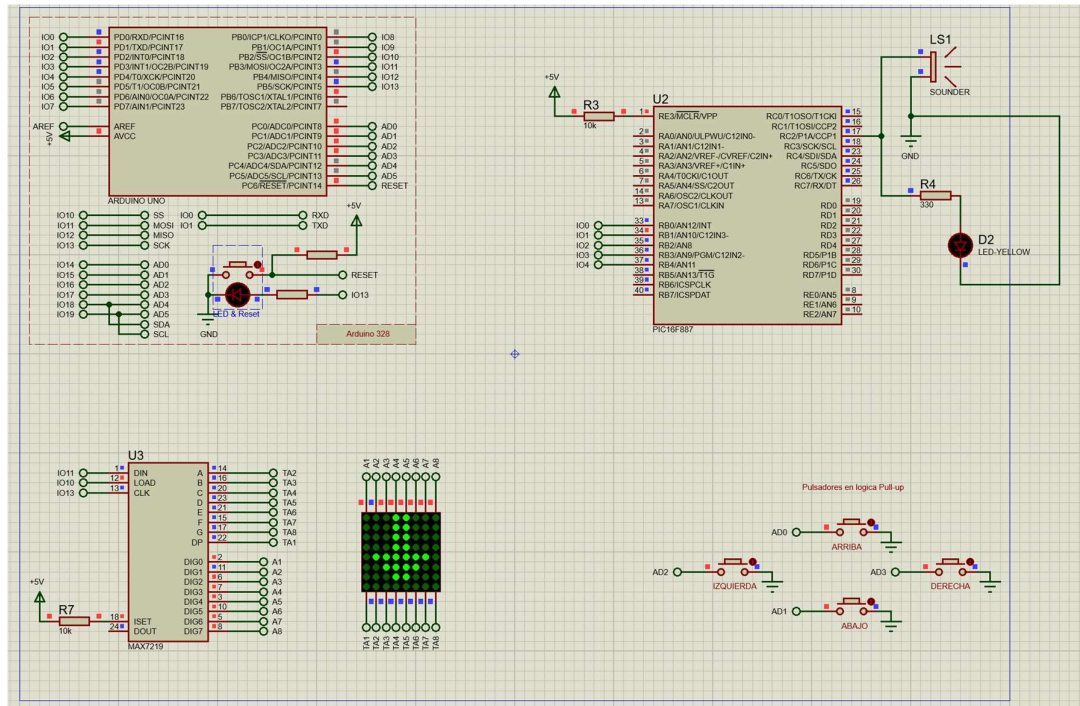


Ilustración 1 - Simulación Proteus

Explicación Del Código Del ATMEGA328P y PIC16F887

ATmega328P – Controlador Maestro del Juego

El **ATmega328P** es el encargado de controlar toda la lógica del juego "Simón Dice". Sus funciones incluyen la lectura de pulsadores, generación de secuencias aleatorias, control de la matriz LED y comunicación con el PIC16F887.

La configuración inicial del microcontrolador se realiza mediante la función:

```
void configurar_puertos() {  
  
    DDRD |= 0B00011111;  
  
    PORTD &= 0B11100000;  
}
```



```
DDRC &= ~((1 << PC0) | (1 << PC1) |
(1 << PC2) | (1 << PC3));

PORTC |= (1 << PC0) | (1 << PC1) |
(1 << PC2) | (1 << PC3);
}
```

En esta sección se configuran los pines del **Puerto C** como entradas para los cuatro pulsadores y los pines del **Puerto D** como salidas para la comunicación con el PIC.

- **Control de la Matriz LED**

La visualización se realiza mediante un controlador **MAX7219**, encargado de manejar la matriz LED de 8×8.

La inicialización se efectúa mediante:

```
void max7219_init() {
    max7219_send(0x09, 0x00);
    max7219_send(0x0A, 0x08);
    max7219_send(0x0B, 0x07);
    max7219_send(0x0C, 0x01);
    max7219_send(0x0F, 0x00);
}
```

Estas instrucciones configuran el brillo, el modo de operación y la cantidad de filas activas de la matriz. Posteriormente, las figuras se muestran mediante:

```
void mostrar_figura(const uint8_t figura[8]) {
    for (uint8_t i = 0; i < 8; i++)
        max7219_send(i + 1, figura[i]);
}
```

```
}
```

Esta función envía cada fila del patrón almacenado en memoria hacia la matriz LED.

- **Generación de Secuencias Aleatorias**

La secuencia del juego se genera utilizando un algoritmo pseudoaleatorio sencillo:

```
uint8_t numero_aleatorio() {  
    static uint8_t semilla = 7;  
    semilla = semilla * 17 + 43;  
    return (semilla % 4) + 1;  
}
```

Cada número generado corresponde a una de las cuatro direcciones posibles del juego. La secuencia completa se almacena mediante:

```
void generar_secuencia_completa() {  
    for (uint8_t i = 0; i < 7; i++) {  
        secuencia[i] = numero_aleatorio();  
    }  
}
```

- **Lectura de Pulsadores**

La detección de las respuestas del usuario se realiza mediante la función:

```
uint8_t leer_boton() {  
  
    if (!(PINC & (1 << PC0))) return 1;  
    if (!(PINC & (1 << PC1))) return 2;  
    if (!(PINC & (1 << PC2))) return 3;
```

```
if (!(PINC & (1 << PC3))) return 4;  
return 0;}
```

Cada botón se asocia con un valor entre 1 y 4, correspondiente a una dirección específica.

- **Comunicación con el PIC16F887**

La comunicación entre ambos microcontroladores se realiza mediante un bus paralelo y una señal de validación denominada **Strobe**.

```
void enviar_comando_pic(uint8_t comando) {  
    PORTD = (PORTD & 0B11110000) |  
            (comando & 0B00001111);  
    PORTD |= (1 << STROBE);  
    _delay_ms(50);  
    PORTD &= ~(1 << STROBE);  
    _delay_ms(50);  
}
```

El ATmega328P coloca el comando en las líneas de datos y genera un pulso en la línea Strobe para indicar al PIC que existe nueva información disponible.

- **Verificación de la Respuesta del Usuario**

Una vez mostrada la secuencia, el sistema verifica cada pulsación realizada por el jugador.

```
if (boton == secuencia[i]) {  
    enviar_comando_pic(5);  
}  
  
else {
```

```
    enviar_comando_pic(6);  
  
    return 0;  
  
}
```

Si la respuesta es correcta se envía el comando 5 al PIC. Si existe un error se envía el comando 6 y se finaliza la partida.

PIC16F887 – Subsistema Generador de Audio

El **PIC16F887** se encarga exclusivamente de la generación de sonidos del juego. Su función principal es recibir los comandos enviados por el **ATmega328P** y reproducir el efecto sonoro correspondiente mediante un buzzer conectado al pin **RC2**.

La salida utilizada para controlar el buzzer se define mediante:

```
#define BUZZER RC2_bit
```

Los sonidos se generan alternando rápidamente el estado lógico del buzzer utilizando retardos en microsegundos. Por ejemplo, la función siguiente produce una frecuencia específica al activar y desactivar repetidamente el buzzer:

```
void beep_500(unsigned int repeticiones) {  
    unsigned int i;  
    for(i = 0; i < repeticiones; i++) {  
        BUZZER = 1;  
        Delay_us(500);  
        BUZZER = 0;  
        Delay_us(500);  
    }  
}
```

```
}
```

A partir de estas funciones básicas se construyen los diferentes efectos sonoros del juego, como el sonido de inicio, cambio de nivel, respuesta correcta, error y victoria.

La comunicación con el ATmega328P se realiza mediante un bus de datos conectado a los pines **RB0-RB3** y una señal de validación (**STROBE**) conectada a **RB4**. Cuando el ATmega activa la línea STROBE, el PIC lee el valor presente en el bus mediante la función:

```
unsigned short leer_comando() {  
    unsigned short valor = 0;  
    if (RB0_bit == 1) valor += 1;  
    if (RB1_bit == 1) valor += 2;  
    if (RB2_bit == 1) valor += 4;  
    if (RB3_bit == 1) valor += 8;  
    return valor;  
}
```

Posteriormente, el comando recibido es interpretado por la función **ejecutar_comando()**, que selecciona el sonido correspondiente:

```
if (dato == 1) sonido_inicio();  
if (dato == 2) sonido_nivel1();  
if (dato == 3) sonido_nivel2();  
if (dato == 4) sonido_nivel3();  
if (dato == 5) sonido_correcto();  
if (dato == 6) sonido_error();  
if (dato == 7) sonido_victoria();
```

Finalmente, dentro del programa principal, el PIC permanece monitoreando la señal STROBE. Cuando detecta un pulso proveniente del ATmega328P, lee el comando, ejecuta el sonido correspondiente y espera a que la señal vuelva a nivel bajo antes de aceptar una nueva instrucción.

```
while(1) {  
  
    if (RB4_bit == 1) {  
  
        comando = leer_comando();  
  
        ejecutar_comando(comando);  
  
        while(RB4_bit == 1);  
  
        Delay_ms(50);  
  
    }  
  
}
```

Gracias a esta arquitectura, el PIC16F887 actúa como un coprocesador dedicado a la síntesis de audio, permitiendo que el ATmega328P se concentre en la lógica del juego y el control de la matriz LED.

Esquema de Comunicación entre microcontroladores

La comunicación entre el **ATmega328P** y el **PIC16F887** se realiza mediante un **bus paralelo unidireccional de 4 bits**, donde el ATmega328P actúa como **maestro** y el PIC16F887 como **esclavo**.

El bus utiliza los pines **PD0, PD1, PD2 y PD3** del ATmega328P para transmitir los comandos, mientras que el pin **PD4** se utiliza como señal de validación (**STROBE**). En el PIC16F887, estas señales son recibidas a través de los pines **RB0, RB1, RB2, RB3 y RB4**, respectivamente.

Cuando el ATmega328P desea reproducir un sonido, coloca primero el valor del comando en las líneas de datos y posteriormente genera un pulso en la línea STROBE mediante la función:

```
void enviar_comando_pic(uint8_t comando) {  
    PORTD = (PORTD & 0B11110000) | (comando & 0B00001111);  
    PORTD |= (1 << STROBE);  
    _delay_ms(50);  
    PORTD &= ~(1 << STROBE);  
    _delay_ms(50);  
}
```

Al detectar el flanco de activación de STROBE, el PIC16F887 lee el estado de las líneas RB0-RB3 utilizando la función:

```
unsigned short leer_comando() {  
    unsigned short valor = 0;
```

```

if (RB0_bit == 1) valor += 1;

if (RB1_bit == 1) valor += 2;

if (RB2_bit == 1) valor += 4;

if (RB3_bit == 1) valor += 8;

return valor;
}

```

Una vez obtenido el valor, el PIC ejecuta la acción correspondiente mediante la función **ejecutar_comando()**, reproduciendo el sonido asociado al evento del juego.

ATmega328P	PIC16F887	Función
PD0	RB0	D0 (Bit 0 del bus de datos)
PD1	RB1	D1 (Bit 1 del bus de datos)
PD2	RB2	D2 (Bit 2 del bus de datos)
PD3	RB3	D3 (Bit 3 del bus de datos)
PD4	RB4	STROBE (Señal de validación y sincronización)

Tabla 2 - Comunicación de Microcontroladores

Este esquema permite una comunicación simple, rápida y confiable entre ambos microcontroladores, garantizando la sincronización entre los eventos visuales generados por el ATmega328P y los efectos sonoros producidos por el PIC16F887.

Enlace de Repositorio

Enlace: <https://n9.cl/hp586>

Conclusiones y Recomendaciones

- **Conclusiones**

1. Se logró implementar satisfactoriamente el juego de memoria "**Simón Dice**" utilizando una arquitectura distribuida basada en dos microcontroladores, donde el **ATmega328P** se encarga de la lógica del juego y el control visual, mientras que el **PIC16F887** administra de forma independiente la generación de audio.
2. La comunicación paralela entre ambos microcontroladores permitió una sincronización adecuada entre los estímulos visuales mostrados en la matriz LED y los efectos sonoros reproducidos por el buzzer, cumpliendo con los requerimientos de funcionamiento en tiempo real del sistema.
3. La utilización de una matriz LED 8×8 controlada mediante el MAX7219 permitió representar de manera clara los símbolos, niveles y estados del juego, mejorando la interacción con el usuario.
4. La implementación de distintos niveles de dificultad demostró la capacidad del sistema para modificar dinámicamente la velocidad y complejidad de las secuencias, incrementando progresivamente el desafío para el jugador.
5. La simulación realizada en Proteus permitió validar el correcto funcionamiento de la lógica de control, la comunicación entre microcontroladores y la generación de efectos sonoros antes de una posible implementación física del proyecto.

- **Recomendaciones**

1. Se recomienda almacenar los archivos **.hex** en rutas cortas y de fácil acceso, evitando carpetas sincronizadas en la nube como **OneDrive** o directorios con nombres extensos, espacios o caracteres especiales. Una ubicación sugerida es:

C:\Proteus\Proyecto_Simon_Matrix

2. Verificar antes de cada simulación que el archivo **.hex** correcto esté asignado a cada microcontrolador. El **ATmega328P** debe contener el programa correspondiente a la lógica del juego y control de la matriz LED, mientras que el **PIC16F887** debe tener cargado el programa encargado de la recepción de comandos y generación de audio.
3. Comprobar que la frecuencia de reloj configurada en Proteus coincida con la utilizada durante la compilación de los programas, ya que una configuración incorrecta puede afectar las temporizaciones, la comunicación entre dispositivos y el comportamiento general del sistema.
4. Asegurar que todos los componentes compartan una misma referencia de tierra (**GND común**). El GND del ATmega328P, PIC16F887, buzzer, matriz LED y fuente de alimentación deben estar interconectados para garantizar una correcta interpretación de las señales digitales.
5. Realizar pruebas individuales de cada subsistema (visual, comunicación y audio) antes de ejecutar el sistema completo, facilitando la detección y corrección de posibles errores durante el proceso de integración.